Contents

1. Objective . . . 3

2. Reference Tools . . . 3

3. Approach . . . 3

4. Architecture . . . 3

4.1 Project files . . . 3

4.2 Cluster architecture . . . 4

4.3 Manifest design . . . 4

4.4 Running the project . . . 4

4.5 Testing and issues observed . . . 5

5. Source Code . . . 5

5.1 kind-config.yaml — Kind cluster definition . . . 5

5.2 00-namespace.yaml — Namespace . . . 5

5.3 01-configmap.yaml — ConfigMap with the web page . . . 5

5.4 02-deployment.yaml — Deployment . . . 6

5.5 03-service.yaml — NodePort Service . . . 6

5.6 deploy_k8s.py — automation script (lines 1–60) . . . 7

5.6 deploy_k8s.py — automation script (lines 61–120) . . . 8

5.6 deploy_k8s.py — automation script (lines 121–157) . . . 9

6. Screenshots and Command Output . . . 10

6.1 Cluster setup . . . 10

6.2 Deployment of the manifests . . . 11

6.3 Pod and service verification . . . 11

6.4 Scaling . . . 13

6.5 Self-healing . . . 14

6.6 Rolling update . . . 14

6.7 Monitoring deployment status . . . 15

7. Results . . . 18

7.1 Deployment summary . . . 18

7.2 Evaluation criteria . . . 18

7.3 Deployment scaling events . . . 19

8. Observations . . . 19

9. Known Limitations . . . 19

10. Conclusion . . . 20

1. Objective

To understand Kubernetes orchestration and deploy containerized applications on a local cluster created with Kind (the task allows Minikube or Kind): set up a multi-node cluster, describe the application declaratively in YAML manifests, deploy sample containers, verify pod and service operation and monitor the deployment status through scaling, self-healing and a rolling update.

2. Reference Tools

Kubernetes is the orchestrator (server v1.37.0). **Kind** (Kubernetes IN Docker, v0.33.0) runs every cluster node as a Docker container, so a realistic multi-node cluster fits on a laptop; **Minikube** is supported by the same script (`--tool minikube`) but Kind was used for this run. **kubectl** v1.37.1 is the command-line client and **Docker Desktop** 29.8.2 on an Apple-silicon Mac is the container runtime. The workload is the official **nginx:1.27-alpine** image, whose home page is replaced by a ConfigMap-provided `index.html` .

3. Approach

1. **Set up a local Kubernetes cluster** — `kind create cluster --name task12 --config kind-config.yaml` builds one control-plane and two worker nodes and maps NodePort 30080 to `localhost:8080` (Figure 1, Figure 2).
2. **Create deployment manifests** — four declarative files in `k8s/` : Namespace, ConfigMap, Deployment (3 replicas, probes, resource limits, rolling-update strategy) and NodePort Service (Sections 5.2–5.5).
3. **Deploy sample containers** — `kubectl apply -f k8s/` creates all objects and `kubectl rollout status` waits for the pods to become available (Figure 3).
4. **Verify pod and service operations** — pod status and placement, Service endpoints, and an HTTP request from both the terminal and a browser (Figure 4–Figure 7).
5. **Monitor deployment status** — scale 3 → 5, delete a pod to test self-healing, update the image, then inspect rollout history, `describe` , logs and events (Figure 9–Figure 14).

The whole procedure is automated by one Python script, `deploy_k8s.py` , which records every command and its real output to `evidence/run_log.json` ; the report figures are generated from that log.

4. Architecture

One Kind cluster, one namespace, one Deployment and one Service. The application files are small and declarative; everything else (cluster creation, verification, scaling, update, monitoring) is driven by the script.

4.1 Project files

File	Role
`kind-config.yaml`	Cluster definition: one control-plane node (with host port mapping 8080 → 30080) and two workers.
`k8s/00-namespace.yaml`	Namespace k8s-demo that isolates all application objects.
`k8s/01-configmap.yaml`	ConfigMap web-content holding index.html, mounted over nginx's html directory.
`k8s/02-deployment.yaml`	Deployment web-app: 3 nginx replicas, probes, requests/limits, RollingUpdate strategy.
`k8s/03-service.yaml`	Service web-service: NodePort 30080 → port 80 of pods labelled app=web-app.
`deploy_k8s.py`	Automation and evidence collection (cluster, deploy, verify, scale, self-heal, update, monitor).
`Task12_Kubernetes.ipynb`	Notebook version of the same workflow (submission alternative to the .py script).

4.2 Cluster architecture

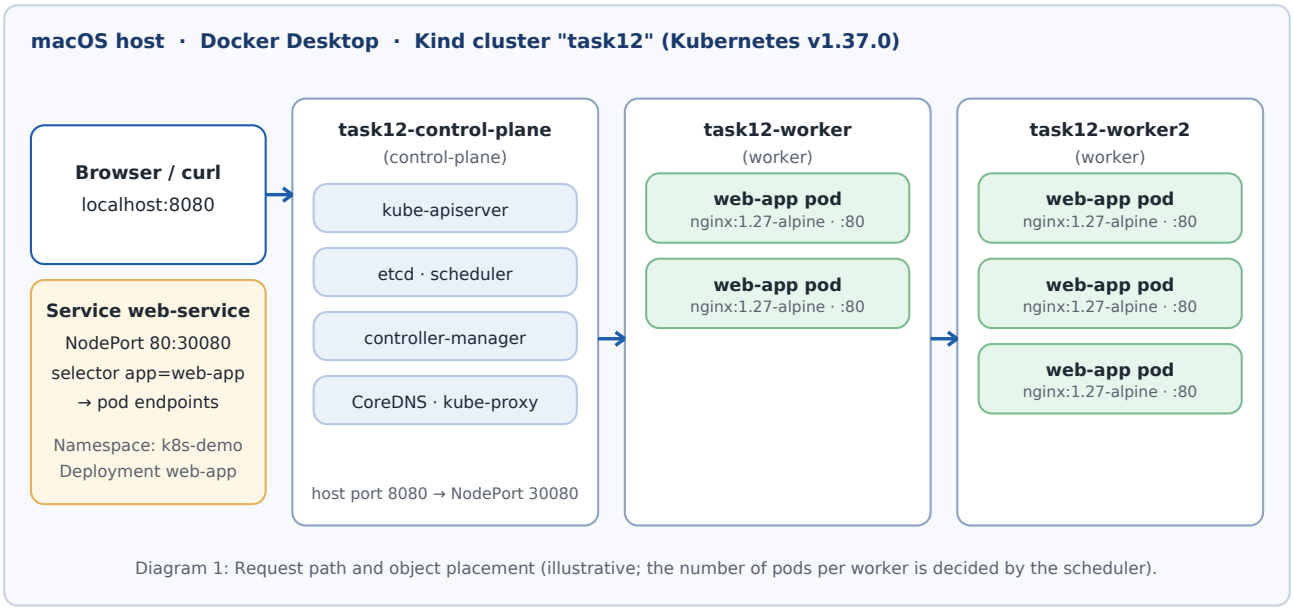


Diagram 1: Request path and object placement (illustrative; the number of pods per worker is decided by the scheduler).

The Service has a stable ClusterIP and load-balances across all ready pods selected by *app=web-app*. Kind publishes the Service's NodePort on the host through *extraPortMappings*, which is why a browser on the Mac can reach *localhost:8080*.

4.3 Manifest design

Setting	Value	Purpose
replicas	3	Desired pod count, restored automatically if a pod is lost.
strategy	RollingUpdate, maxSurge 1, maxUnavailable 0	Image updates replace pods one at a time with no capacity loss.
readinessProbe	HTTP GET / :80, delay 3 s, period 5 s	A pod receives traffic only once nginx answers.
livenessProbe	HTTP GET / :80, delay 10 s, period 10 s	Restarts a hung container.
resources	requests 50m / 32Mi · limits 200m / 128Mi	Scheduling hint and protection against noisy neighbours.
ConfigMap volume	web-content → /usr/share/nginx/html	Page content is configuration, not baked into the image.
Service	NodePort 30080 → 80	Stable endpoint and load balancing, reachable from the host.

4.4 Running the project

1. `python deploy_k8s.py`
2. `open http://localhost:8080`
3. `python build_report.py # builds the PDF from evidence/`

Prerequisites: Docker Desktop running, `kubectl`, `kind` and Python 3 with Pillow. The script exits with a clear message if a tool is missing and re-uses an existing `task12` cluster, so it can be re-run safely (`kubectl apply` is idempotent). `--cleanup` deletes the cluster afterwards.

4.5 Testing and issues observed

Observation	Cause	Outcome
FailedScheduling warnings on all three pods at the start (0/3 nodes available, untolerated taint)	The manifests were applied immediately after cluster creation, while the two workers were still NotReady and carried the not-ready taint.	Transient: the pods were scheduled about one second later once a worker became Ready (Figure 14).
`kubectl top pods` → "Metrics API not available"	metrics-server is not installed in a default Kind cluster.	Expected; documented as a limitation. The script treats this step as non-fatal.
"v1 Endpoints is deprecated in v1.33+" warning	kubectl get endpoints uses the legacy API; EndpointSlice replaces it.	Harmless; the endpoint list is still correct.
First pods all placed on task12-worker2	Most likely worker2 became Ready first, so the scheduler used it.	No impact on availability; later pods may use either worker.

5. Source Code

Every file needed to reproduce the deployment is listed below with line numbers. The five YAML files are the Kubernetes deliverables; `deploy_k8s.py` (157 lines) is the Python source that builds the cluster, applies the manifests and records the evidence. Lines longer than 118 characters are wrapped and continued with "↳".

5.1 `kind-config.yaml` — Kind cluster definition

```
1  kind: Cluster
2  apiVersion: kind.x-k8s.io/v1alpha4
3  nodes:
4    - role: control-plane
5      extraPortMappings:
6        - containerPort: 30080   # NodePort of web-service
7          hostPort: 8080         # http://localhost:8080
8          protocol: TCP
9    - role: worker
10   - role: worker
```

5.2 `00-namespace.yaml` — Namespace

```
1  apiVersion: v1
2  kind: Namespace
3  metadata:
4    name: k8s-demo
5    labels:
6      project: task12
```

5.3 `01-configmap.yaml` — ConfigMap with the web page

```
1  apiVersion: v1
2  kind: ConfigMap
3  metadata:
4    name: web-content
5    namespace: k8s-demo
6  data:
7    index.html: |
8      <!DOCTYPE html>
9      <html>
10       <head><title>Task 12 - Kubernetes Demo</title></head>
11       <body style="font-family: sans-serif; text-align: center; margin-top: 80px;">
12         <h1>Hello from Kubernetes!</h1>
13         <p>Task 12: Cluster Setup and Deployment</p>
14         <p>Served by an nginx pod managed by a Deployment.</p>
15       </body>
16     </html>
```

5.4 `02-deployment.yaml` — Deployment

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
  namespace: k8s-demo
  labels:
    app: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web-app
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  template:
    metadata:
      labels:
        app: web-app
    spec:
      containers:
        - name: nginx
          image: nginx:1.27-alpine
          ports:
            - containerPort: 80
          resources:
            requests:
              cpu: 50m
              memory: 32Mi
            limits:
              cpu: 200m
              memory: 128Mi
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 5
          livenessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 10
            periodSeconds: 10
          volumeMounts:
            - name: web-content
              mountPath: /usr/share/nginx/html
      volumes:
        - name: web-content
          configMap:
            name: web-content
```

5.5 `03-service.yaml` — NodePort Service

```yaml
apiVersion: v1
kind: Service
metadata:
  name: web-service
  namespace: k8s-demo
spec:
  type: NodePort
  selector:
    app: web-app
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080
```

5.6 `deploy_k8s.py` — automation script (lines 1–60)

```
#!/usr/bin/env python3
"""Task 12 - Kubernetes cluster setup and deployment (Kind or Minikube).

Usage:
    python deploy_k8s.py --tool kind        # default
    python deploy_k8s.py --tool minikube
    python deploy_k8s.py --cleanup          # delete cluster afterwards

Every command and its real output is saved to evidence/run_log.json and
evidence/run_log.txt, and rendered as terminal-style PNG screenshots in
evidence/ (needs: pip install pillow).
"""
import argparse, json, shutil, subprocess, sys, time, urllib.request
from pathlib import Path

ROOT = Path(__file__).parent
EVID = ROOT / "evidence"
EVID.mkdir(exist_ok=True)
NS = "k8s-demo"
CLUSTER = "task12"
LOG = []  # list of {"title", "cmd", "output"}


def run(title, cmd, check=True, timeout=600):
    """Run a shell command, print + record its output."""
    print(f"\n=== {title} ===\n$ {cmd}")
    p = subprocess.run(cmd, shell=True, capture_output=True, text=True, timeout=timeout)
    out = (p.stdout + p.stderr).strip()
    print(out)
    LOG.append({"title": title, "cmd": cmd, "output": out, "rc": p.returncode})
    if check and p.returncode != 0:
        save_logs()
        sys.exit(f"Command failed ({p.returncode}): {cmd}")
    return out


def save_logs():
    (EVID / "run_log.json").write_text(json.dumps(LOG, indent=2))
    with open(EVID / "run_log.txt", "w") as f:
        for e in LOG:
            f.write(f"=== {e['title']} ===\n$ {e['cmd']}\n{e['output']}\n\n")
    try:
        render_screenshots()
    except ImportError:
        print("Pillow not installed - skipping PNG rendering (pip install pillow)")


def render_screenshots():
    """Render each recorded command as a terminal-style PNG."""
    from PIL import Image, ImageDraw, ImageFont
    font = None
    for f in ("DejaVuSansMono.ttf", "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
              "Menlo.ttc", "Consolas.ttf", "Courier New.ttf"):
        try:
            font = ImageFont.truetype(f, 15); break
        except OSError:
            continue
    font = font or ImageFont.load_default()
    for i, e in enumerate(LOG, 1):
        lines = [f"$ {e['cmd']}"] + e["output"].splitlines()[:45]
```

5.6 `deploy_k8s.py` — automation script (lines 61–120)

```
        lines = [l[:130] for l in lines]
        w = max(900, 9 * max(len(l) for l in lines) + 40)
        h = 40 + 21 * len(lines)
        img = Image.new("RGB", (w, h), (24, 26, 31))
        d = ImageDraw.Draw(img)
        d.rectangle([0, 0, w, 26], fill=(50, 54, 62))
        d.text((10, 5), e["title"], fill=(220, 220, 220), font=font)
        for n, l in enumerate(lines):
            d.text((14, 36 + 21 * n), l, fill=(120, 220, 140) if n == 0 else (225, 225, 225), font=font)
        img.save(EVID / f"{i:02d}_{e['title'].lower().replace(' ', '_').replace('/', '-')[:40]}.png")


def create_cluster(tool):
    if tool == "kind":
        existing = run("List kind clusters", "kind get clusters", check=False)
        if CLUSTER not in existing.split():
            run("Create Kind cluster (1 control-plane + 2 workers)",
                f"kind create cluster --name {CLUSTER} --config kind-config.yaml --wait 120s")
        run("Set kubectl context", f"kubectl config use-context kind-{CLUSTER}")
    else:
        run("Start Minikube cluster", f"minikube start -p {CLUSTER} --nodes=2 --driver=docker")
        run("Set kubectl context", f"kubectl config use-context {CLUSTER}")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--tool", choices=["kind", "minikube"], default="kind")
    ap.add_argument("--cleanup", action="store_true", help="delete the cluster at the end")
    a = ap.parse_args()

    # 1. Prerequisites
    for t in ("docker", "kubectl", a.tool):
        if not shutil.which(t):
            sys.exit(f"'{t}' not found in PATH - install it first (see README.md)")
    run("Tool versions", f"docker --version && kubectl version --client && {a.tool} version")

    # 2. Cluster setup
    create_cluster(a.tool)
    run("Cluster info", "kubectl cluster-info")
    run("Cluster nodes", "kubectl get nodes -o wide")

    # 3. Deploy manifests
    run("Apply manifests", "kubectl apply -f k8s/")
    run("Wait for rollout", f"kubectl rollout status deployment/web-app -n {NS} --timeout=180s")

    # 4. Verify pods and services
    run("Get pods", f"kubectl get pods -n {NS} -o wide")
    run("Get deployment, service, endpoints", f"kubectl get deploy,svc,endpoints,cm -n {NS}")
    run("Describe service", f"kubectl describe svc web-service -n {NS}")

    # 5. Functional test
    if a.tool == "kind":
        url = "http://localhost:8080"
    else:
        url = run("Minikube service URL", f"minikube service web-service -n {NS} -p {CLUSTER} --url").splitlines()[-1]
    time.sleep(3)
    try:
        body = urllib.request.urlopen(url, timeout=10).read().decode()
        LOG.append({"title": "HTTP test", "cmd": f"curl {url}", "output": body.strip(), "rc": 0})
        print(f"\nHTTP test OK ({url}):\n{body}")
```

5.6 `deploy_k8s.py` — automation script (lines 121–157)

```
    except Exception as ex:
        LOG.append({"title": "HTTP test", "cmd": f"curl {url}", "output": f"FAILED: {ex}", "rc": 1})
        print("HTTP test failed:", ex)

    # 6. Scaling + self-healing
    run("Scale to 5 replicas", f"kubectl scale deployment/web-app --replicas=5 -n {NS}")
    run("Wait after scale", f"kubectl rollout status deployment/web-app -n {NS} --timeout=120s")
    run("Pods after scaling", f"kubectl get pods -n {NS}")
    victim = run("Pick a pod", f"kubectl get pods -n {NS} -o jsonpath='{{.items[0].metadata.name}}'").strip("'")
    run("Delete a pod (self-healing test)", f"kubectl delete pod {victim} -n {NS}")
    run("Wait for recovery", f"kubectl rollout status deployment/web-app -n {NS} --timeout=120s")
    run("Pods after self-healing", f"kubectl get pods -n {NS}")

    # 7. Rolling update
    run("Rolling update to nginx 1.27.3-alpine",
        f"kubectl set image deployment/web-app nginx=nginx:1.27.3-alpine -n {NS}")
    run("Rollout status", f"kubectl rollout status deployment/web-app -n {NS} --timeout=180s")
    run("Rollout history", f"kubectl rollout history deployment/web-app -n {NS}")

    # 8. Monitoring deployment status
    run("Deployment details", f"kubectl describe deployment web-app -n {NS}")
    run("Pod logs", f"kubectl logs deployment/web-app -n {NS} --tail=10")
    run("Recent events", f"kubectl get events -n {NS} --sort-by=.lastTimestamp")
    run("Resource usage (needs metrics-server)", f"kubectl top pods -n {NS}", check=False)

    save_logs()
    print(f"\nDone. Logs + screenshots saved in {EVID}")

    if a.cleanup:
        if a.tool == "kind":
            subprocess.run(f"kind delete cluster --name {CLUSTER}", shell=True)
        else:
            subprocess.run(f"minikube delete -p {CLUSTER}", shell=True)


if __name__ == "__main__":
    main()
```

6. Screenshots and Command Output

All terminal figures below show the exact output captured by `deploy_k8s.py` during the run on 07 October 2026 (`evidence/run_log.json`); they are rendered from that log in a terminal style, not retyped. Figure 7 and Figure 8 are real screen captures. Long outputs are split over two images and are marked as parts. Pod names and IPs are those of this particular run.

6.1 Cluster setup

Tool versions

```
$ docker --version && kubectl version --client && kind version
Docker version 29.8.2, build 7fc2dff
Client Version: v1.37.1
Kustomize Version: v5.8.1
kind v0.33.0 go1.27.0 darwin/arm64
```

List kind clusters

```
$ kind get clusters
desktop
```

Create Kind cluster (1 control-plane + 2 workers)

```
$ kind create cluster --name task12 --config kind-config.yaml --wait 120s
Creating cluster "task12" ...
 • Ensuring node image (kindest/node:v1.37.0) □  ...
 ✓ Ensuring node image (kindest/node:v1.37.0) □
 • Preparing nodes □ □ □   ...
 ✓ Preparing nodes □ □ □
 • Writing configuration □  ...
 ✓ Writing configuration □
 • Starting control-plane □  ...
 ✓ Starting control-plane □
 • Installing CNI □  ...
 ✓ Installing CNI □
 • Installing StorageClass □  ...
 ✓ Installing StorageClass □
 • Joining worker nodes □  ...
 ✓ Joining worker nodes □
 • Waiting ≤ 2m0s for control-plane = Ready □  ...
 ✓ Waiting ≤ 2m0s for control-plane = Ready □
 • Ready after 9s □
Set kubectl context to "kind-task12"
You can now use your cluster with:

kubectl cluster-info --context kind-task12

Have a question, bug, or feature request? Let us know! https://kind.sigs.k8s.io/#community □
```

Figure 1: Tool versions, existing Kind clusters (only Docker Desktop's own "desktop" cluster) and creation of the three-node cluster task12 with kind-config.yaml.

Set kubectl context

```
$ kubectl config use-context kind-task12
Switched to context "kind-task12".
```

Cluster info

```
$ kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:50875
CoreDNS is running at https://127.0.0.1:50875/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
```

Cluster nodes

```
$ kubectl get nodes -o wide
NAME                STATUS     ROLES           AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE                       KERNEL-VERSION
              CONTAINER-RUNTIME
task12-control-plane   Ready      control-plane   21s   v1.37.0   172.18.0.5    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
       containerd://2.3.4
task12-worker          NotReady   <none>          11s   v1.37.0   172.18.0.7    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
       containerd://2.3.4
task12-worker2         NotReady   <none>          11s   v1.37.0   172.18.0.6    <none>        Debian GNU/Linux 13 (trixie)   7.0.14-linuxkit (arm64)
       containerd://2.3.4
```

Figure 2: kubectl context switched to kind-task12; cluster-info shows the control plane and CoreDNS; the three nodes immediately after creation (workers still NotReady, 11 s old).

6.2 Deployment of the manifests

Apply manifests

```
$ kubectl apply -f k8s/
namespace/k8s-demo created
configmap/web-content created
deployment.apps/web-app created
service/web-service created
```

Wait for rollout

```
$ kubectl rollout status deployment/web-app -n k8s-demo --timeout=180s
Waiting for deployment "web-app" rollout to finish: 0 of 3 updated replicas are available...
Waiting for deployment "web-app" rollout to finish: 1 of 3 updated replicas are available...
Waiting for deployment "web-app" rollout to finish: 2 of 3 updated replicas are available...
deployment "web-app" successfully rolled out
```

Figure 3: kubectl apply -f k8s/ creates the Namespace, ConfigMap, Deployment and Service, and rollout status waits until all 3 replicas are available.

6.3 Pod and service verification

Get pods

```
$ kubectl get pods -n k8s-demo -o wide
NAME                       READY   STATUS    RESTARTS   AGE   IP           NODE             NOMINATED NODE   READINESS GATES
web-app-54c6b7ccf5-9zb9t   1/1     Running   0          20s   10.244.2.3   task12-worker2   <none>           <none>
web-app-54c6b7ccf5-d447v   1/1     Running   0          20s   10.244.2.2   task12-worker2   <none>           <none>
web-app-54c6b7ccf5-nlcgx   1/1     Running   0          20s   10.244.2.4   task12-worker2   <none>           <none>
```

Get deployment, service, endpoints

```
$ kubectl get deploy,svc,endpoints,cm -n k8s-demo
NAME                     READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/web-app   3/3     3            3           20s

NAME                  TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/web-service   NodePort   10.96.204.129   <none>        80:30080/TCP   20s

NAME                    ENDPOINTS                                      AGE
endpoints/web-service   10.244.2.2:80,10.244.2.3:80,10.244.2.4:80   20s

NAME                         DATA   AGE
configmap/kube-root-ca.crt   1      20s
configmap/web-content        1      20s
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
```

Figure 4: Pods (3 × Running 1/1 with IPs and nodes) and the Deployment, Service, Endpoints and ConfigMaps in the k8s-demo namespace.

Describe service

```
$ kubectl describe svc web-service -n k8s-demo
Name:                     web-service
Namespace:                k8s-demo
Labels:                   <none>
Annotations:              <none>
Selector:                 app=web-app
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.96.204.129
IPs:                      10.96.204.129
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30080/TCP
Endpoints:                10.244.2.3:80,10.244.2.2:80,10.244.2.4:80
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

Figure 5: kubectl describe svc web-service: NodePort 30080 → targetPort 80, selector app=web-app and the three pod endpoints.

HTTP test

```
$ curl http://localhost:8080
<!DOCTYPE html>
<html>
  <head><title>Task 12 - Kubernetes Demo</title></head>
  <body style="font-family: sans-serif; text-align: center; margin-top: 80px;">
    <h1>Hello from Kubernetes!</h1>
    <p>Task 12: Cluster Setup and Deployment</p>
    <p>Served by an nginx pod managed by a Deployment.</p>
  </body>
</html>
```

Figure 6: HTTP test from the terminal: curl http://localhost:8080 returns the custom page served from the ConfigMap.

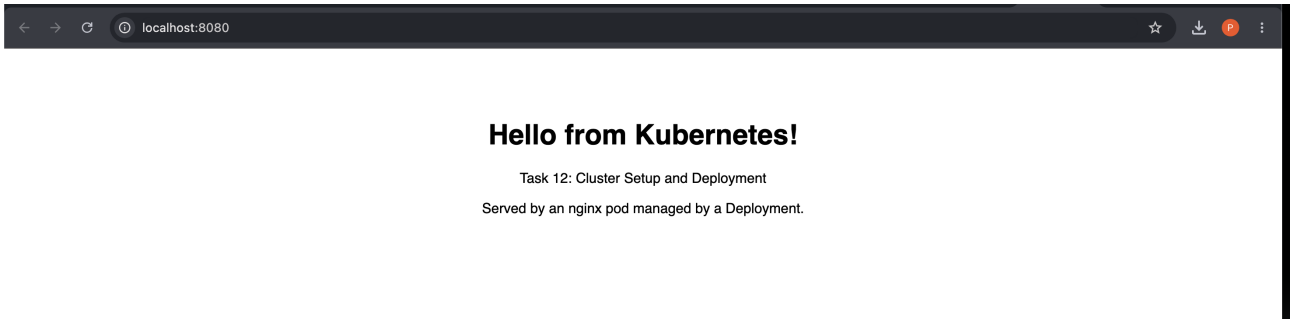


Figure 7: The application opened in Chrome at http://localhost:8080 (blank lower part of the capture cropped).

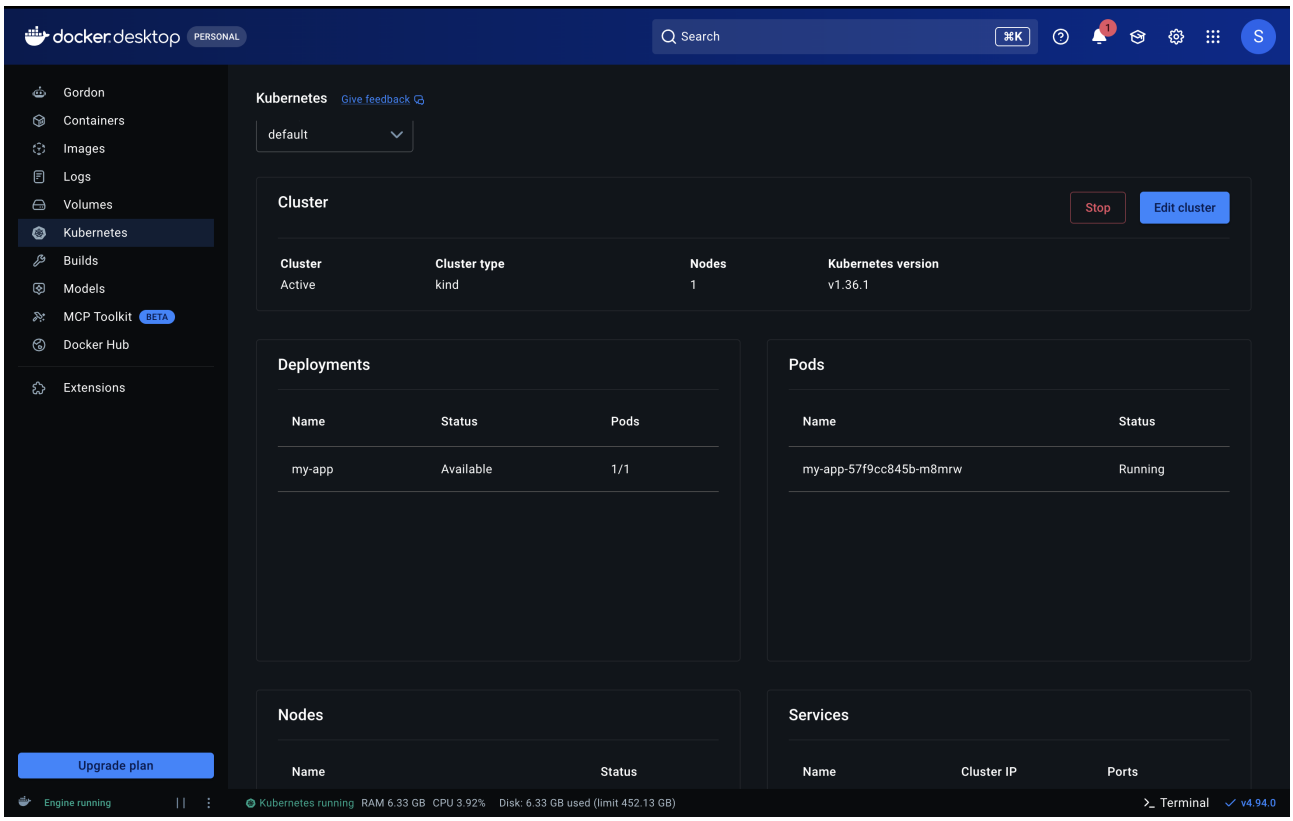


Figure 8: Docker Desktop → Kubernetes view. This is Docker Desktop's own built-in single-node Kind cluster ("desktop", running an unrelated my-app deployment), shown for the environment. The task12 cluster runs separately as three Kind node containers.

6.4 Scaling

Scale to 5 replicas

```
$ kubectl scale deployment/web-app --replicas=5 -n k8s-demo
deployment.apps/web-app scaled
```

Wait after scale

```
$ kubectl rollout status deployment/web-app -n k8s-demo --timeout=120s
Waiting for deployment "web-app" rollout to finish: 3 of 5 updated replicas are available...
Waiting for deployment "web-app" rollout to finish: 4 of 5 updated replicas are available...
deployment "web-app" successfully rolled out
```

Pods after scaling

```
$ kubectl get pods -n k8s-demo
NAME                       READY   STATUS    RESTARTS   AGE
web-app-54c6b7ccf5-9zb9t   1/1     Running   0          44s
web-app-54c6b7ccf5-d447v   1/1     Running   0          44s
web-app-54c6b7ccf5-nlcgx   1/1     Running   0          44s
web-app-54c6b7ccf5-sclnw   1/1     Running   0          21s
web-app-54c6b7ccf5-vk6sq   1/1     Running   0          21s
```

Figure 9: Scaling web-app from 3 to 5 replicas: command, rollout wait and the resulting 5 Running pods.

6.5 Self-healing

Pick a pod

```
$ kubectl get pods -n k8s-demo -o jsonpath='{.items[0].metadata.name}'
web-app-54c6b7ccf5-9zb9t
```

Delete a pod (self-healing test)

```
$ kubectl delete pod web-app-54c6b7ccf5-9zb9t -n k8s-demo
pod "web-app-54c6b7ccf5-9zb9t" deleted from k8s-demo namespace
```

Wait for recovery

```
$ kubectl rollout status deployment/web-app -n k8s-demo --timeout=120s
Waiting for deployment "web-app" rollout to finish: 4 of 5 updated replicas are available...
deployment "web-app" successfully rolled out
```

Pods after self-healing

```
$ kubectl get pods -n k8s-demo
NAME                       READY   STATUS    RESTARTS   AGE
web-app-54c6b7ccf5-d447v   1/1     Running   0          50s
web-app-54c6b7ccf5-nlcgx   1/1     Running   0          50s
web-app-54c6b7ccf5-sclnw   1/1     Running   0          27s
web-app-54c6b7ccf5-vk6sq   1/1     Running   0          27s
web-app-54c6b7ccf5-zjdts   1/1     Running   0          6s
```

Figure 10: Self-healing test: the first pod (…-9zb9t) is deleted, the Deployment recovers to 5 available replicas and a new pod (…-zjdts, 6 s old) appears.

6.6 Rolling update

Rolling update to nginx 1.27.3-alpine

```
$ kubectl set image deployment/web-app nginx=nginx:1.27.3-alpine -n k8s-demo
deployment.apps/web-app image updated
```

Rollout status

```
$ kubectl rollout status deployment/web-app -n k8s-demo --timeout=180s
Waiting for deployment "web-app" rollout to finish: 1 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 1 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 1 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 2 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 2 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 2 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 2 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 3 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 3 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 3 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 3 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 4 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 4 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 4 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 4 out of 5 new replicas have been updated...
Waiting for deployment "web-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "web-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "web-app" rollout to finish: 1 old replicas are pending termination...
deployment "web-app" successfully rolled out
```

Rollout history

```
$ kubectl rollout history deployment/web-app -n k8s-demo
deployment.apps/web-app
REVISION  CHANGE-CAUSE
1         <none>
2         <none>
```

Figure 11: Rolling update to nginx:1.27.3-alpine: the new ReplicaSet grows one pod at a time (maxSurge 1) until the old pods are terminated; rollout history shows revisions 1 and 2.

6.7 Monitoring deployment status

Deployment details

```
$ kubectl describe deployment web-app -n k8s-demo
Name:                   web-app
Namespace:              k8s-demo
CreationTimestamp:      Wed, 07 Oct 2026 21:50:27 +0530
Labels:                 app=web-app
Annotations:            deployment.kubernetes.io/revision: 2
Selector:               app=web-app
Replicas:               5 desired | 5 updated | 5 total | 5 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  0 max unavailable, 1 max surge
Pod Template:
  Labels:  app=web-app
  Containers:
   nginx:
    Image:      nginx:1.27.3-alpine
    Port:       80/TCP
    Host Port:  0/TCP
    Limits:
      cpu:     200m
      memory:  128Mi
    Requests:
      cpu:        50m
      memory:     32Mi
    Liveness:     http-get http://:80/ delay=10s timeout=1s period=10s successThreshold=1 failureThreshold=3
    Readiness:    http-get http://:80/ delay=3s timeout=1s period=5s successThreshold=1 failureThreshold=3
    Environment:  <none>
    Mounts:
      /usr/share/nginx/html from web-content (rw)
```

Figure 12 (part 1 of 2): kubectl describe deployment web-app after the update: revision 2, 5/5 replicas, RollingUpdate 0 max unavailable / 1 max surge, image nginx:1.27.3-alpine, probes, limits, ConfigMap volume and the two ReplicaSets.

Deployment details

```
$ kubectl describe deployment web-app -n k8s-demo
  Volumes:
   web-content:
    Type:          ConfigMap (a volume populated by a ConfigMap)
    Name:          web-content
    Optional:      false
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  web-app-54c6b7ccf5 (0/0 replicas created)
NewReplicaSet:   web-app-74999cdbdb (5/5 replicas created)
Events:
  Type    Reason             Age              From                   Message
  ----    ------             ----             ----                   -------
  Normal  ScalingReplicaSet  98s              deployment-controller  Scaled up replica set web-app-54c6b7ccf5 from 0 to 3
  Normal  ScalingReplicaSet  75s              deployment-controller  Scaled up replica set web-app-54c6b7ccf5 from 3 to 5
  Normal  ScalingReplicaSet  48s              deployment-controller  Scaled up replica set web-app-74999cdbdb from 0 to 1
  Normal  ScalingReplicaSet  33s              deployment-controller  Scaled down replica set web-app-54c6b7ccf5 from 5 to 4
  Normal  ScalingReplicaSet  33s              deployment-controller  Scaled up replica set web-app-74999cdbdb from 1 to 2
  Normal  ScalingReplicaSet  18s              deployment-controller  Scaled down replica set web-app-54c6b7ccf5 from 4 to 3
  Normal  ScalingReplicaSet  18s              deployment-controller  Scaled up replica set web-app-74999cdbdb from 2 to 3
  Normal  ScalingReplicaSet  12s              deployment-controller  Scaled down replica set web-app-54c6b7ccf5 from 3 to 2
  Normal  ScalingReplicaSet  12s              deployment-controller  Scaled up replica set web-app-74999cdbdb from 3 to 4
  Normal  ScalingReplicaSet  0s (x3 over 6s)  deployment-controller  (combined from similar events): Scaled down replica set web-app-54c6b7ccf5 from 1
     to 0
```

Figure 12 (part 2 of 2): continued.

Pod logs

```
$ kubectl logs deployment/web-app -n k8s-demo --tail=10
10.244.1.1 - - [07/Oct/2026:16:21:31 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:21:36 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:21:37 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:21:42 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:21:46 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:21:47 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:21:51 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:21:56 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:21:57 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
10.244.1.1 - - [07/Oct/2026:16:22:01 +0000] "GET / HTTP/1.1" 200 321 "-" "kube-probe/1.37" "-"
Found 6 pods, using pod/web-app-74999cdbdb-rsskh
```

Resource usage (needs metrics-server)

```
$ kubectl top pods -n k8s-demo
error: Metrics API not available
```

Figure 13: nginx pod logs (readiness/liveness probe requests returning 200) and kubectl top pods, which reports that the Metrics API is not available (no metrics-server installed).

Recent events

```
$ kubectl get events -n k8s-demo --sort-by=.lastTimestamp
LAST SEEN   TYPE      REASON              OBJECT                          MESSAGE
98s         Normal    ScalingReplicaSet   deployment/web-app              Scaled up replica set web-app-54c6b7ccf5 from 0 to 3
98s         Normal    SuccessfulCreate    replicaset/web-app-54c6b7ccf5   Created pod: web-app-54c6b7ccf5-nlcgx
98s         Normal    SuccessfulCreate    replicaset/web-app-54c6b7ccf5   Created pod: web-app-54c6b7ccf5-9zb9t
98s         Normal    SuccessfulCreate    replicaset/web-app-54c6b7ccf5   Created pod: web-app-54c6b7ccf5-d447v
98s         Warning   FailedScheduling    pod/web-app-54c6b7ccf5-9zb9t    0/3 nodes are available: 3 node(s) had untolerated taint(s). preemption: 0/3
    nodes are available: 3 Preemption is not helpful for scheduling.
98s         Warning   FailedScheduling    pod/web-app-54c6b7ccf5-d447v    0/3 nodes are available: 3 node(s) had untolerated taint(s). preemption: 0/3
    nodes are available: 3 Preemption is not helpful for scheduling.
98s         Warning   FailedScheduling    pod/web-app-54c6b7ccf5-nlcgx    0/3 nodes are available: 3 node(s) had untolerated taint(s). preemption: 0/3
    nodes are available: 3 Preemption is not helpful for scheduling.
97s         Normal    Scheduled           pod/web-app-54c6b7ccf5-9zb9t    Successfully assigned k8s-demo/web-app-54c6b7ccf5-9zb9t to task12-worker2
97s         Normal    Pulling             pod/web-app-54c6b7ccf5-9zb9t    Pulling image "nginx:1.27-alpine"
97s         Normal    Scheduled           pod/web-app-54c6b7ccf5-d447v    Successfully assigned k8s-demo/web-app-54c6b7ccf5-d447v to task12-worker2
97s         Normal    Pulling             pod/web-app-54c6b7ccf5-d447v    Pulling image "nginx:1.27-alpine"
97s         Normal    Pulling             pod/web-app-54c6b7ccf5-nlcgx    Pulling image "nginx:1.27-alpine"
97s         Normal    Scheduled           pod/web-app-54c6b7ccf5-nlcgx    Successfully assigned k8s-demo/web-app-54c6b7ccf5-nlcgx to task12-worker2
87s         Normal    Started             pod/web-app-54c6b7ccf5-9zb9t    Container started
87s         Normal    Created             pod/web-app-54c6b7ccf5-9zb9t    Container created
87s         Normal    Pulled              pod/web-app-54c6b7ccf5-9zb9t    Successfully pulled image "nginx:1.27-alpine" in 9.546s (9.547s including
    waiting). Image size: 21832241 bytes.
86s         Normal    Started             pod/web-app-54c6b7ccf5-d447v    Container started
86s         Normal    Created             pod/web-app-54c6b7ccf5-d447v    Container created
86s         Normal    Pulled              pod/web-app-54c6b7ccf5-d447v    Successfully pulled image "nginx:1.27-alpine" in 1.386s (10.898s including
    waiting). Image size: 21832241 bytes.
84s         Normal    Pulled              pod/web-app-54c6b7ccf5-nlcgx    Successfully pulled image "nginx:1.27-alpine" in 1.384s (12.275s including
    waiting). Image size: 21832241 bytes.
84s         Normal    Created             pod/web-app-54c6b7ccf5-nlcgx    Container created
84s         Normal    Started             pod/web-app-54c6b7ccf5-nlcgx    Container started
75s         Normal    SuccessfulCreate    replicaset/web-app-54c6b7ccf5   Created pod: web-app-54c6b7ccf5-vk6sq
75s         Normal    ScalingReplicaSet   deployment/web-app              Scaled up replica set web-app-54c6b7ccf5 from 3 to 5
75s         Normal    SuccessfulCreate    replicaset/web-app-54c6b7ccf5   Created pod: web-app-54c6b7ccf5-sclnw
75s         Normal    Scheduled           pod/web-app-54c6b7ccf5-vk6sq    Successfully assigned k8s-demo/web-app-54c6b7ccf5-vk6sq to task12-worker
75s         Normal    Pulling             pod/web-app-54c6b7ccf5-vk6sq    Pulling image "nginx:1.27-alpine"
75s         Normal    Scheduled           pod/web-app-54c6b7ccf5-sclnw    Successfully assigned k8s-demo/web-app-54c6b7ccf5-sclnw to task12-worker
75s         Normal    Pulling             pod/web-app-54c6b7ccf5-sclnw    Pulling image "nginx:1.27-alpine"
65s         Normal    Created             pod/web-app-54c6b7ccf5-sclnw    Container created
65s         Normal    Started             pod/web-app-54c6b7ccf5-sclnw    Container started
65s         Normal    Pulled              pod/web-app-54c6b7ccf5-sclnw    Successfully pulled image "nginx:1.27-alpine" in 9.537s (9.537s including
    waiting). Image size: 21832241 bytes.
60s         Normal    Pulled              pod/web-app-54c6b7ccf5-vk6sq    Successfully pulled image "nginx:1.27-alpine" in 4.833s (14.338s including
    waiting). Image size: 21832241 bytes.
60s         Normal    Started             pod/web-app-54c6b7ccf5-vk6sq    Container started
60s         Normal    Created             pod/web-app-54c6b7ccf5-vk6sq    Container created
54s         Normal    Started             pod/web-app-54c6b7ccf5-zjdts    Container started
54s         Normal    Scheduled           pod/web-app-54c6b7ccf5-zjdts    Successfully assigned k8s-demo/web-app-54c6b7ccf5-zjdts to task12-worker
54s         Normal    Killing             pod/web-app-54c6b7ccf5-9zb9t    Stopping container nginx
54s         Normal    Created             pod/web-app-54c6b7ccf5-zjdts    Container created
54s         Normal    Pulled              pod/web-app-54c6b7ccf5-zjdts    Container image "nginx:1.27-alpine" already present on machine and can be
    accessed by the pod
54s         Normal    SuccessfulCreate    replicaset/web-app-54c6b7ccf5   Created pod: web-app-54c6b7ccf5-zjdts
48s         Normal    SuccessfulCreate    replicaset/web-app-74999cdbdb   Created pod: web-app-74999cdbdb-rsskh
48s         Normal    ScalingReplicaSet   deployment/web-app              Scaled up replica set web-app-74999cdbdb from 0 to 1
```

Figure 14 (part 1 of 2): Namespace events sorted by time: scheduling, image pulls, scale-up, pod deletion and the rolling update.

Recent events

```
$ kubectl get events -n k8s-demo --sort-by=.lastTimestamp
48s         Normal    Pulling             pod/web-app-74999cdbdb-rsskh    Pulling image "nginx:1.27.3-alpine"
48s         Normal    Scheduled           pod/web-app-74999cdbdb-rsskh    Successfully assigned k8s-demo/web-app-74999cdbdb-rsskh to task12-worker
39s         Normal    Started             pod/web-app-74999cdbdb-rsskh    Container started
39s         Normal    Created             pod/web-app-74999cdbdb-rsskh    Container created
39s         Normal    Pulled              pod/web-app-74999cdbdb-rsskh    Successfully pulled image "nginx:1.27.3-alpine" in 9.016s (9.016s including
    waiting). Image size: 21565101 bytes.
33s         Normal    Pulling             pod/web-app-74999cdbdb-ggdjp    Pulling image "nginx:1.27.3-alpine"
33s         Normal    SuccessfulDelete    replicaset/web-app-54c6b7ccf5   Deleted pod: web-app-54c6b7ccf5-zjdts
33s         Normal    ScalingReplicaSet   deployment/web-app              Scaled up replica set web-app-74999cdbdb from 1 to 2
33s         Normal    ScalingReplicaSet   deployment/web-app              Scaled down replica set web-app-54c6b7ccf5 from 5 to 4
33s         Normal    Killing             pod/web-app-54c6b7ccf5-zjdts    Stopping container nginx
33s         Normal    Scheduled           pod/web-app-74999cdbdb-ggdjp    Successfully assigned k8s-demo/web-app-74999cdbdb-ggdjp to task12-worker2
33s         Normal    SuccessfulCreate    replicaset/web-app-74999cdbdb   Created pod: web-app-74999cdbdb-ggdjp
24s         Normal    Pulled              pod/web-app-74999cdbdb-ggdjp    Successfully pulled image "nginx:1.27.3-alpine" in 8.9s (8.9s including
    waiting). Image size: 21565101 bytes.
24s         Normal    Created             pod/web-app-74999cdbdb-ggdjp    Container created
24s         Normal    Started             pod/web-app-74999cdbdb-ggdjp    Container started
18s         Normal    Scheduled           pod/web-app-74999cdbdb-lwrlm    Successfully assigned k8s-demo/web-app-74999cdbdb-lwrlm to task12-worker2
18s         Normal    Killing             pod/web-app-54c6b7ccf5-nlcgx    Stopping container nginx
18s         Normal    Created             pod/web-app-74999cdbdb-lwrlm    Container created
18s         Normal    Started             pod/web-app-74999cdbdb-lwrlm    Container started
18s         Normal    ScalingReplicaSet   deployment/web-app              Scaled down replica set web-app-54c6b7ccf5 from 4 to 3
18s         Normal    SuccessfulCreate    replicaset/web-app-74999cdbdb   Created pod: web-app-74999cdbdb-lwrlm
18s         Normal    SuccessfulDelete    replicaset/web-app-54c6b7ccf5   Deleted pod: web-app-54c6b7ccf5-nlcgx
18s         Normal    Pulled              pod/web-app-74999cdbdb-lwrlm    Container image "nginx:1.27.3-alpine" already present on machine and can be
    accessed by the pod
18s         Normal    ScalingReplicaSet   deployment/web-app              Scaled up replica set web-app-74999cdbdb from 2 to 3
12s         Normal    SuccessfulDelete    replicaset/web-app-54c6b7ccf5   Deleted pod: web-app-54c6b7ccf5-sclnw
12s         Normal    Pulled              pod/web-app-74999cdbdb-vs9sz    Container image "nginx:1.27.3-alpine" already present on machine and can be
    accessed by the pod
12s         Normal    Created             pod/web-app-74999cdbdb-vs9sz    Container created
12s         Normal    Started             pod/web-app-74999cdbdb-vs9sz    Container started
12s         Normal    Scheduled           pod/web-app-74999cdbdb-vs9sz    Successfully assigned k8s-demo/web-app-74999cdbdb-vs9sz to task12-worker
12s         Normal    ScalingReplicaSet   deployment/web-app              Scaled down replica set web-app-54c6b7ccf5 from 3 to 2
12s         Normal    Killing             pod/web-app-54c6b7ccf5-sclnw    Stopping container nginx
12s         Normal    SuccessfulCreate    replicaset/web-app-74999cdbdb   Created pod: web-app-74999cdbdb-vs9sz
12s         Normal    ScalingReplicaSet   deployment/web-app              Scaled up replica set web-app-74999cdbdb from 3 to 4
6s          Normal    Scheduled           pod/web-app-74999cdbdb-6jfst    Successfully assigned k8s-demo/web-app-74999cdbdb-6jfst to task12-worker
6s          Normal    Killing             pod/web-app-54c6b7ccf5-vk6sq    Stopping container nginx
6s          Normal    Started             pod/web-app-74999cdbdb-6jfst    Container started
6s          Normal    Created             pod/web-app-74999cdbdb-6jfst    Container created
6s          Normal    Pulled              pod/web-app-74999cdbdb-6jfst    Container image "nginx:1.27.3-alpine" already present on machine and can be
    accessed by the pod
6s          Normal    SuccessfulCreate    replicaset/web-app-74999cdbdb   Created pod: web-app-74999cdbdb-6jfst
6s          Normal    SuccessfulDelete    replicaset/web-app-54c6b7ccf5   Deleted pod: web-app-54c6b7ccf5-vk6sq
0s          Normal    SuccessfulDelete    replicaset/web-app-54c6b7ccf5   Deleted pod: web-app-54c6b7ccf5-d447v
0s          Normal    Killing             pod/web-app-54c6b7ccf5-d447v    Stopping container nginx
0s          Normal    ScalingReplicaSet   deployment/web-app              (combined from similar events): Scaled down replica set web-app-54c6b7ccf5
    from 1 to 0
```

Figure 14 (part 2 of 2): continued.

7. Results

The deployment succeeded and every verification step passed. The values below are taken from the run log; the figures referenced show the original output.

7.1 Deployment summary

Item	Result	Evidence
Tools	Docker 29.8.2 · kubectl client v1.37.1 · kind v0.33.0 (darwin/arm64)	Figure 1
Cluster	task12: 3 nodes (1 control-plane, 2 workers), Kubernetes v1.37.0, all registered	Figure 1, Figure 2
Objects created	Namespace k8s-demo, ConfigMap web-content, Deployment web-app, Service web-service	Figure 3
Initial rollout	0 → 3 replicas available; "successfully rolled out"; pods Running 1/1, 0 restarts	Figure 3, Figure 4
Service	NodePort, ClusterIP 10.96.204.129, 80:30080/TCP, 3 endpoints	Figure 4, Figure 5
HTTP test	localhost:8080 returned the custom "Hello from Kubernetes!" page (terminal and Chrome)	Figure 6, Figure 7
Scaling	3 → 5 replicas; 5 pods Running after about 20 s	Figure 9
Self-healing	Deleted web-app-54c6b7ccf5-9zb9t; replacement web-app-54c6b7ccf5-zjdts created; 5/5 available again	Figure 10
Rolling update	nginx:1.27-alpine → nginx:1.27.3-alpine; revision 2; new ReplicaSet 5/5, old ReplicaSet 0/0	Figure 11, Figure 12
Probes / logs	kube-probe GET / requests answered 200 every few seconds	Figure 13
Resource usage	kubectl top: Metrics API not available (metrics-server not installed)	Figure 13

7.2 Evaluation criteria

Criterion	How it was met	Status
Deployment success	All 3 replicas available and serving HTTP; Service has 3 endpoints; scale and update completed with "successfully rolled out".	Met
YAML configuration accuracy	All manifests applied without error; describe output confirms the intended image, probes, limits, strategy and ConfigMap volume; Service selector matches the pod labels.	Met
Operational understanding	Scaling, self-healing, rolling update, logs, events and rollout history were exercised and interpreted (Section 8).	Met

7.3 Deployment scaling events

Age	Event (from kubectl describe deployment)
98s	Scaled up replica set web-app-54c6b7ccf5 from 0 to 3
75s	Scaled up replica set web-app-54c6b7ccf5 from 3 to 5
48s	Scaled up replica set web-app-74999cdbdb from 0 to 1
33s	Scaled down replica set web-app-54c6b7ccf5 from 5 to 4
33s	Scaled up replica set web-app-74999cdbdb from 1 to 2
18s	Scaled down replica set web-app-54c6b7ccf5 from 4 to 3
18s	Scaled up replica set web-app-74999cdbdb from 2 to 3
12s	Scaled down replica set web-app-54c6b7ccf5 from 3 to 2
12s	Scaled up replica set web-app-74999cdbdb from 3 to 4
0s (x3 over 6s)	Scaled down replica set web-app-54c6b7ccf5 from 1 to 0

The first two rows are the initial deployment (0 → 3) and the manual scale-up (3 → 5) of the original ReplicaSet. During the update, each step then adds one pod to the new ReplicaSet `web-app-74999cdbdb` and removes one from the old `web-app-54c6b7ccf5`; ages are measured at the moment `describe` was run, so smaller means later. The pod-log command also reported 6 pods matching the Deployment during the update (5 desired plus 1 surge), consistent with `maxSurge: 1`.

8. Observations

Declarative model. The manifests state the desired state and the controllers converge on it: *kubectl apply* created all four objects and the Deployment brought up three ready pods without any manual container handling.

Layered ownership. Deployment → ReplicaSet → Pods. The image change created a second ReplicaSet (revision 2) while the first was scaled to 0/0 and kept, which is what makes rollback possible.

Self-healing. Deleting `web-app-54c6b7ccf5-9zb9t` was followed within seconds by a new pod (`…-zjdts`); the Deployment returned to 5/5 available without intervention.

Capacity-preserving update. The strategy (maxUnavailable 0, maxSurge 1) plus readiness probes made the rollout progress 1 → 2 → 3 → 4 → 5 new pods while old pods were terminated afterwards; the final state shows 5 available and 0 unavailable.

Service abstraction. The Service's three endpoints are the pod IPs; because it selects by label, newly created or replacement pods join automatically and terminated pods leave, so the URL never changes.

Scheduling behaviour. The initial FailedScheduling warnings were a start-up race (nodes not yet Ready), not a misconfiguration; Kubernetes retried and scheduled the pods about a second later.

Monitoring tools. get / describe / logs / events / rollout status / rollout history answer different questions (state, configuration, application output, history of decisions, progress). Resource metrics need metrics-server in addition.

Environment. Docker Desktop also hosts its own single-node cluster (Figure 8); Kind's task12 cluster is independent of it, and kubectl contexts decide which one a command targets (switched explicitly in Figure 2).

9. Known Limitations

• **Single machine.** All nodes are containers on one laptop, so node failure, real networking and cloud load balancers are not exercised.

• **No metrics-server.** CPU and memory usage (*kubectl top*) and autoscaling were not available.

- **NodePort via port mapping.** External access depends on Kind's host port mapping (8080); a second cluster using the same port would conflict.
- **Internet dependency.** The nginx images are pulled from Docker Hub on first use.
- **No persistence or TLS.** The page is static ConfigMap content; there is no storage, Ingress or HTTPS.
- **Change-cause history.** Rollout history shows <none> for CHANGE-CAUSE because no change-cause annotation was set.

10. Conclusion

A three-node Kubernetes cluster was created locally with Kind, a containerized nginx application was deployed through declarative Namespace, ConfigMap, Deployment and Service manifests, verified through pod, endpoint, curl and browser checks, scaled from 3 to 5 replicas, shown to self-heal after a pod deletion and upgraded from nginx 1.27 to 1.27.3 with a rolling update configured for zero unavailability. The captured output confirms every deliverable: the YAML files (Section 5), the deployment screenshots (Section 6) and this cluster setup report.